Lab Outcomes:

- **Object-Oriented Programming (OOP) Concepts**: Understand the basics of classes, objects, and inheritance in C#, including creating classes, deriving child classes, and overriding methods.
- **Understand Polymorphism**: Learn how polymorphism allows methods to behave differently based on the object type, enabling flexibility in code.
- **Implement Inheritance and Method Overriding**: Learn how inheritance and method overriding help modify behavior in derived classes, supporting dynamic polymorphism.

OOP in C#

Classes and objects are the two main aspects of object-oriented programming. So, a class is a template for objects, and an object is an instance of a class.

When the individual objects are created, they inherit all the variables and methods from the class.

Example:

```
class Vehicle
{
  string color = "black";
  static void Main(string[] args)
  {
    Car myObj1 = new Vehicle();
    Car myObj2 = new Vehicle();
    Console.WriteLine(myObj1.color);
    Console.WriteLine(myObj2.color);
  }
}
```

Inheritance

In C#, it is possible to inherit fields and methods from one class to another. We group the "inheritance concept" into two categories:

- **Derived Class** (child) - the class that inherits from another class
- **Base Class** (parent) - the class being inherited from

To inherit from a class, use the : symbol.

C# Polymorphism

Polymorphism is one of the features provided by Object Oriented Programming. Polymorphism simply means occurring in more than one form.

Types of Polymorphism

There are two types of polymorphism:

1. Compile Time Polymorphism / Static Polymorphism
2. Run-Time Polymorphism / Dynamic Polymorphism

1. Compile Time Polymorphism

In compile time polymorphism, the compiler identifies which method is being called at the compile time.

In C#, we achieve compile time polymorphism through 2 ways:

- Method overloading
- Operator overloading

Method overloading

In a C# class, we can create methods with the same name in a class if they have:

- different numbers of parameter
- types of parameters

Example

```
void totalSum() {...}

void totalSum(int a) {...}

void totalSum(int a, int b) {...}

void totalSum(float a, float b) {...}
```

Example

```
// method adds two integer numbers
void totalSum(int a, int b)
{
   Console.WriteLine("The sum of numbers is " + (a + b));
}

// method adds two double-type numbers
// totalSum() method is overloaded
void totalSum(double a, double b)
{
   Console.WriteLine("The sum of numbers is " + (a + b));
}

static void Main(string[] args)
{

   Program sum1 = new Program();
   sum1.totalSum(5, 7);
   sum1.totalSum(53.5, 8.7);
}
```

2. Runtime Polymorphism

In runtime polymorphism, the method that is called is determined at the runtime not at compile time.

The runtime polymorphism is achieved by:

- Method Overriding

Method Overriding in C#

During inheritance in C#, if the same method is present in both the superclass and the subclass. Then, the method in the subclass overrides the same method in the superclass. This is called method overriding.

In this case, the same method will perform one operation in the superclass and another operation in the subclass.

We can use virtual and override keywords to achieve method overriding.

Example

```
class Polygon
{
   // method to render a shape
   public virtual void render()
   {
      Console.WriteLine("Rendering Polygon...");
   }

}

class Square : Polygon
{
   // overriding render() method
   public override void render()
```

```
  {
    Console.WriteLine("Rendering Square...");
  }

}
class myProgram
{
  public static void Main()
  {
    // obj1 is the object of Polygon class
    Polygon obj1 = new Polygon();

    // calls render() method of Polygon Superclass
    obj1.render();

    // here, obj1 is the object of derived class Square
    obj1 = new Square();

    // calls render() method of derived class Square
    obj1.render();
  }
}
```

C# Abstraction

Data Abstraction is the property by virtue of which only the essential details are exhibited to the user. The trivial or the non-essentials units aren't exhibited to the user.

Abstract Classes

- An abstract class is declared with the help of **abstract** keyword.
- In C#, you are **not** allowed to create objects of the abstract class. Or in other words, you cannot use the abstract class directly with the new operator.
- Class that contains the abstract keyword with some of its methods (not all abstract method) is known as an **Abstract Base Class**.

- Class that contains the abstract keyword with all of its methods is known as pure Abstract Base Class.

Example:

```
abstract class Polygon
{
   // Making render() an abstract method
   public abstract void render();
}

class Square : Polygon
{
   // Implementing the abstract method render() for Square
   public override void render()
   {
      Console.WriteLine("Rendering Square...");
   }
}

class Triangle : Polygon
{
   // Implementing the abstract method render() for Triangle
   public override void render()
   {
      Console.WriteLine("Rendering Triangle...");
   }
}

class Program
{
   public static void Main()
   {
      // You cannot instantiate an abstract class
      // So, the following line is incorrect:
      // Polygon obj1 = new Polygon();
```

```
    // Instead, you work with instances of derived classes
    Polygon square = new Square();
    Polygon triangle = new Triangle();

    // Calls render() method of Square
    square.render();

    // Calls render() method of Triangle
    triangle.render();
  }
```

Interfaces in C#

Interface in C# is a blueprint of a class. It is like abstract class because all the methods which are declared inside the interface are abstract methods. It cannot have method body and cannot be instantiated.

Its implementation must be provided by class or struct. The class or struct which implements the interface, must provide the implementation of all the methods declared inside the interface.

```
interface IPolygon
{
   // Interface method
   void render();
}


// Implement the IPolygon interface in Square class
class Square : IPolygon
{
   // Implementing the render method for Square
```

```
    public void render()
    {
        Console.WriteLine("Rendering Square...");
    }
}

// Implement the IPolygon interface in Triangle class
class Triangle : IPolygon
{
    // Implementing the render method for Triangle
    public void render()
    {
        Console.WriteLine("Rendering Triangle...");
    }
}

class Program
{
    public static void Main()
    {
        // Instantiate Square and Triangle, which implement IPolygon
        IPolygon square = new Square();
        IPolygon triangle = new Triangle();

        // Calls render() method of Square
        square.render();
```

```
    // Calls render() method of Triangle
    triangle.render();
}
```

Tasks:

Objective:

To implement a small Library Management System in C# using Object-Oriented Programming (OOP) principles.

Requirements:

The system should allow for managing books, members, and borrow/return actions in a library.

Task 1: Define Classes and Relationships,

i). Create a Book Class

Properties:

- Title (string)
- Author (string)
- ISBN (string, unique for each book)
- IsAvailable (bool, initially set to true)

Methods:

- DisplayInfo() - Displays book details.

ii). Create a Member Class

Properties:

- MemberID (int, unique for each member)
- Name (string)
- MaxBooks (int, maximum books a member can borrow, set default as 5)

Methods:

- DisplayInfo() - Displays member details.

Task 2: Implement Inheritance and Polymorphism

Create Student and Faculty Classes that inherit from Member

- **Student:** Set MaxBooks to 3.
- **Faculty**: Set MaxBooks to 10.
- Override DisplayInfo() to add specific information (e.g., type of member: Student or Faculty).

Task 3: Implement Library Class (Composition)

Create a Library Class

Properties:

- Books (List of Book objects)
- Members (List of Member objects)

Methods:

- AddBook(Book book): Adds a new book to the library.
- AddMember(Member member): Adds a new member.
- BorrowBook(string ISBN, int MemberID): Allows a member to borrow a book if available and within their limit.
- ReturnBook(string ISBN, int MemberID): Allows a member to return a borrowed book.

Task 4: Encapsulation and Validation

- Encapsulate the Library Management System
- Ensure only valid data can be entered (e.g., MemberID and ISBN must be unique).
- When borrowing a book, check if the book is available and the member is within their MaxBooks limit.
- Display error messages if a book is unavailable or if a member exceeds their borrowing limit.

Task 5: Implement Main Program

- Test the System
- In the Main method, create an instance of Library and add some sample Books and Members.
- Allow members to borrow and return books.
- Display the current status of all books and members.

Lab Assessment Criteria:
Lab #05 Marks distribution

	ER1	ER2	ER8
Task	2 points	2 points	4 points

Lab # 05

Rubric Evaluation Guideline:

#	**Qualities & Criteria**	**0 < Poor <=4**	**4.1 < Satisfactory <= 7**	**7.1 < Excellent <=10**
ER1	Task/Program Completion	Minimal or no task functionality was achieved	Some tasks were completed but few has errors or incomplete functionalities.	All tasks were completed without any errors.
#	**Qualities & Criteria**	**0 < Poor <=4**	**4.1 < Satisfactory <= 7**	**7.1 < Excellent <=10**
ER2	Code Structure and Design	Poor code structure and missing essential code elements like functions, data structures, or control flow.	Code functions but lacks proper structure, documentation, or uses inefficient algorithms.	Well-structured, documented code using appropriate data structures and algorithms.
#	**Qualities & Criteria**	**0 < Poor <= 4**	**4.1< Satisfactory <= 7**	**7.1< Excellent <= 10**
ER8	Question & Answer	Answers some questions but not confidently or based on lab task knowledge.	Answers most questions confidently and based on lab task knowledge.	Answers all questions confidently and demonstrates a deep understanding of the given lab task.